

including CPU, memory and other infrastructure requirements. This helps organisations optimise Kubernetes environments, maintain performance and ensure effective resource management.

Kubernetes environments are generally divided into three categories:

- **Self-managed Kubernetes:** Organisations build and manage the entire Kubernetes environment, including the control plane and worker nodes, using virtual machines or cloud instances. This provides maximum control but requires greater operational responsibility.
- **Managed Kubernetes:** The cloud provider manages the control plane, while organisations focus on managing worker nodes, node pools and applications. This reduces operational complexity while maintaining control over the application environment.
- **Autopilot Kubernetes:** The cloud provider manages the complete Kubernetes infrastructure, including resource provisioning and cluster management, allowing organisations to focus only on deploying and running applications. For example, Google Kubernetes Engine (GKE) provides both Standard mode, where users manage the Kubernetes environment, and Autopilot mode, where Google Cloud handles infrastructure management.

Along with containerization, modern cloud-native environments rely on CI/CD, Infrastructure as Code (IaC) and observability platforms. IaC enables automated infrastructure provisioning, with tools such as Terraform widely used for this. The observability ecosystem, supported by communities such as CNCF, includes a wide range of tools for logging, tracing, metrics and cost engineering, including Fluentd, Fluent Bit, Dynatrace,

Datadog, Splunk, CloudWatch and cloud monitoring platforms.

These tools collect signals generated by applications, users and services, helping organisations monitor, analyse and improve system performance. Platforms such as Google Cloud Operations Suite combine services like Cloud Monitoring, Cloud Trace, Cloud Logging, Error Reporting and Cloud Debugger to provide an integrated observability platform.

Building an observability data pipeline

Observability begins with collecting signals generated from multiple sources, including applications, APIs and microservices. These signals are processed through collectors and monitoring tools such as Prometheus, Grafana and operational agents. In Google Cloud, collected data can be stored and processed through services such as Cloud Storage, BigQuery and Pub/Sub. Cloud Storage provides low-cost storage, while BigQuery is used for advanced analysis of transactional and operational data. The choice of storage and processing platform depends on the organisation's requirements, use cases and existing infrastructure.

Managing observability data also requires effective routing and governance. Google Cloud's Cloud Logging service centralises logs, while Cloud Logs Router helps direct data to appropriate destinations, including Cloud Storage, BigQuery or third-party platforms such as AWS S3. The overall observability pipeline involves instrumenting applications, collecting telemetry data, processing it through appropriate tools and generating customised dashboards for analysis. With advancements in AIOps, backend

processes are increasingly being enhanced with AI-driven automation, reducing manual intervention and improving system management.

Observability tools are evolving rapidly, with platforms such as Google Looker, Grafana, and others enabling advanced analytics and system visibility. Over the next few years, observability is expected to become increasingly integrated with AI, with OpenTelemetry configurations, APIs and SDKs moving towards an API-driven approach.

AI-driven observability enables systems to automatically analyse issues and take corrective actions, leading to agent-based environments where intelligent agents understand system behaviour, perform tasks and respond to failures. Alongside AI integration, organisations are also focusing on cost engineering, compliance, security, and governance while building observability solutions.

Observability must also integrate with SRE and DevOps workflows, including CI/CD pipelines, to improve deployment frequency and reduce deployment time. For Kubernetes adoption, organisations need to focus on reliability, scalability and automated telemetry pipelines that collect signals from multiple sources.

Major practices include defining SLOs (Service Level Objectives) to establish reliability targets and using SLIs (Service Level Indicators) to measure performance against those objectives. Technologies such as service mesh and eBPF, a kernel-based technology for performance tuning and deeper system visibility, are also becoming important.

Automation remains central to lowering the manual processes and strengthening DevOps-driven operations. **END** 🐧

This article is based on the talk 'Navigating Kubernetes Challenges with Observability Insights' by Saurabh Mishra, DevOps lead, Global Payments, delivered at Open Source India 2025. It has been transcribed and curated by Saba Aafreen of the EFY Group..